SUBWAY RIDERSHIP · FORECASTING

Predicting the next hour of the *subway*.

A station-by-station, hour-by-hour ridership model for the MTA, built on roughly two years of fare-tap data spanning 2022–2024.

AUTHOR
ESH S. CHHURA

REPOSITORY
GITHUB.COM/ESHCHHURA

FINAL MODEL
LIGHTGBM · R^2 0.981

What's in here.

01	Executive summary	02
02	The problem and the client	03
03	Data source and structure	04
04	Data wrangling	05
05	Exploratory findings	06
06	Feature engineering	08
07	Train and test split	09
08	Model comparison	10
09	Backtesting	12
10	Final test results	13

EXECUTIVE SUMMARY

I built a model that predicts hourly ridership at every MTA subway station from a 51-million-row Kaggle release of fare-tap records spanning May 2022 through May 2024. The final model, a LightGBM regressor, achieves an R^2 of 0.981 on a held-out test set with a mean absolute error of 36.4 riders per station-hour.

Tree-based models outperformed the linear baselines by a wide margin. Linear models stayed around R^2 0.90 because the relationship between ridership and time is strongly nonlinear, with separate morning and evening peaks on weekdays that flatten into a single broad ridge on weekends. A LightGBM with 127 leaves captured that shape and held it across three time-ordered backtest folds with little drift between folds.

The problem, and who I built this for.

Why this matters

The MTA moves several million riders a day, and the people who plan train frequencies and station staffing are essentially making bets about how many riders will show up where, and when. Most of those decisions are made from historical averages and experience-based planning assumptions. I wanted to see how far a model trained on roughly two years of OMNY and MetroCard taps could push that.

The question I asked

> *Given a station and a specific hour, can I predict how many riders will tap in, accurately enough that the MTA could actually plan around the answer?*

Who the client is

I framed this for the MTA's operations and planning team, specifically the people who decide things like how many trains to send through a busy line at 5 PM on a Tuesday, or where to position a station agent during a holiday week. The audience cares about three things: forecast accuracy at the station-hour grain, behavior at the peaks, and stability over time. Stability matters because nobody wants to retrain a model every Monday.

51M rows
HOURLY TAP RECORDS

431
STATION IDS

~2 yrs
MAY 2022 → MAY 2024

5
BOROUGHS COVERED

"Forecasting hourly ridership at the station level is the kind of problem where averages are not enough. A Tuesday at a midtown hub is not a Tuesday at a quiet outerborough station."

The data, and where it came from.

Source

The base dataset is the `MTA Subway Hourly Ridership 2022–2024` set on Kaggle, originally pulled from the MTA's public open-data portal. Every row represents one station, one hour, one fare class, and one payment method, which means a single hour at a busy station can produce a dozen or more rows depending on how riders paid.

What's in a row

Twelve columns total. The useful raw fields include the timestamp, station identifier, borough, payment method (OMNY or MetroCard), fare class category, ridership count, and transfers count. The final model uses station, borough, calendar, lag, and rolling ridership features. Latitude, longitude, and the redundant geo-string were dropped during aggregation since station_id captures station identity directly.

Scale

Approximately 51.2 million rows spanning May 2022 through May 2024. After cleaning, aggregation, and lag-feature creation, the modeling table has 7,542,884 station-hour rows.

SCHEMA · RAW

`transit_timestamp`	datetime
`transit_mode`	str
`station_complex_id`	str
`station_complex`	str
`borough`	str
`payment_method`	str
`fare_class_category`	str
`ridership`	int
`transfers`	int
`latitude`	float
`longitude`	float
`Georeference`	str

Cleaning, wrangling, and the bits I cut.

Loading 4.6 GB within local memory constraints

The first thing pandas reported was that several columns had mixed types, and the dataset required memory-conscious processing. I downcasted integer columns, parsed the timestamp once and kept it as a proper datetime, and dropped the redundant `Georeference` string since latitude and longitude already carried that information.

Renaming for clarity

I renamed columns so I would not have to type `station_complex_id` repeatedly. The schema after this step uses `timestamp`, `station_id`, `station`, and `payment_type`.

Aggregating to station-hour

The grain I wanted to predict at is one row per station per hour, not one row per fare class. I rolled up by timestamp and station_id, summing ridership while preserving station name and borough as station-level attributes. That collapse, combined with the lag-feature step that follows, brings the table down to 7,542,884 station-hour rows.

Main cleaning steps

The main cleaning steps were timestamp parsing, column renaming, memory-conscious dtype handling, aggregation from fare-class rows to station-hour rows, and removal of leakage columns before modeling. The cleaned data contains 431 unique station IDs. After lag creation and the chronological training split, 428 station IDs appear in the training encoder.

Payment-method columns retained

I retained MetroCard and OMNY ridership columns in the cleaned wide table, which allowed me to analyze payment-method behavior separately from the final forecasting model. That side data feeds the OMNY chart on the next page. It is not part of the modeling feature set.

Exploratory findings from the data.

The day has two peaks, and the weekend smooths them out

The clearest signal in the entire dataset is the daily commute pattern. On weekdays there is a sharp morning peak around 8 AM, a midday valley, and a slightly larger evening peak around 5 PM. On weekends, both peaks disappear and the curve becomes a single broad ridge from late morning to early evening that mostly tracks leisure traffic. Any model that ignores hour-of-day will struggle, which is essentially what the linear baselines turned out to do.

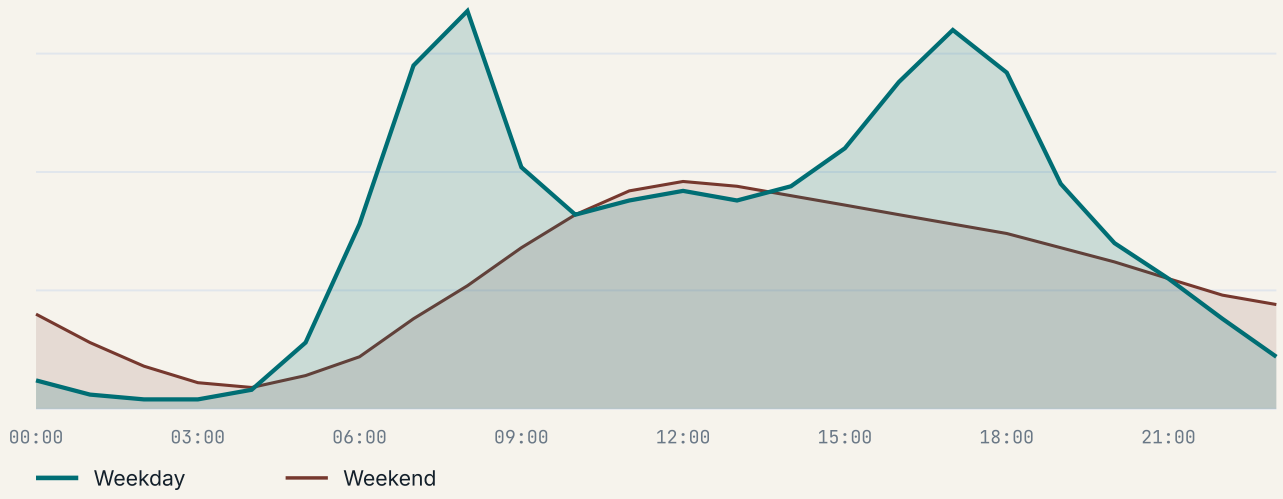

FIG 01 *Mean ridership per station, by hour of day, split into weekday and weekend. The weekday curve shows the two-peak shape characteristic of a commuter system. The weekend curve is gentler and shifted later in the day. Stylized – exact hourly values not computed in EDA.*

Where the riders are, and how they pay.

Manhattan dominates volume, but not as completely as expected

Manhattan accounts for the largest share of hourly ridership volume, with Brooklyn and Queens splitting most of the remainder. Staten Island barely registers at the subway grain, which makes sense given that most of its transit is the ferry and the SIR. The EDA shows clear borough-level differences. Since station ID captures most local variation, borough is best understood as a coarse location feature rather than the main location signal.

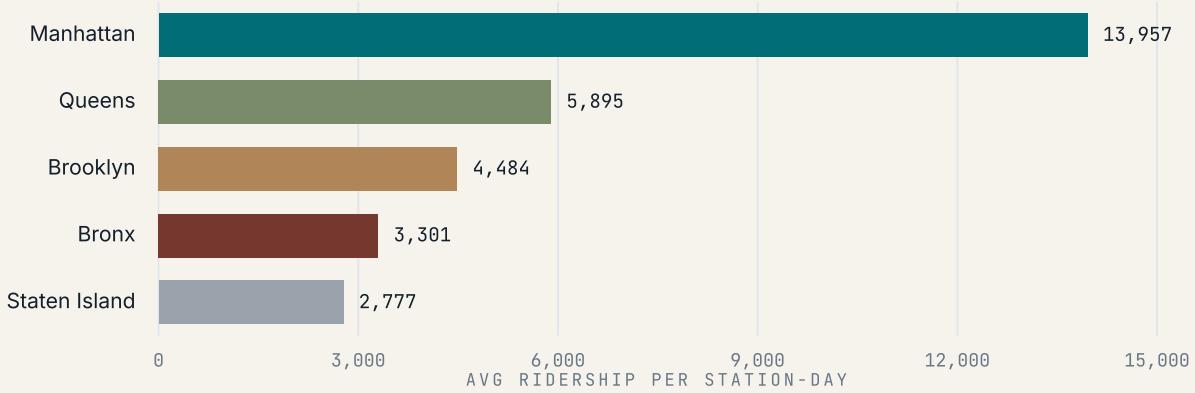

FIG 02 *Average ridership per station-day by borough, based on the EDA notebook's borough-level aggregation. Manhattan stations carry the highest average per-station daily volume.*

OMNY adoption varies sharply by borough

OMNY share differs nearly twofold across boroughs, with Manhattan at 46.7% and Staten Island at 24.2%. The rank order does not perfectly track ridership volume — Brooklyn (39.6%) shows higher OMNY adoption than Queens (35.0%) despite Queens having higher per-station ridership. The split is informative for downstream policy work, even though it does not feed the ridership model directly.

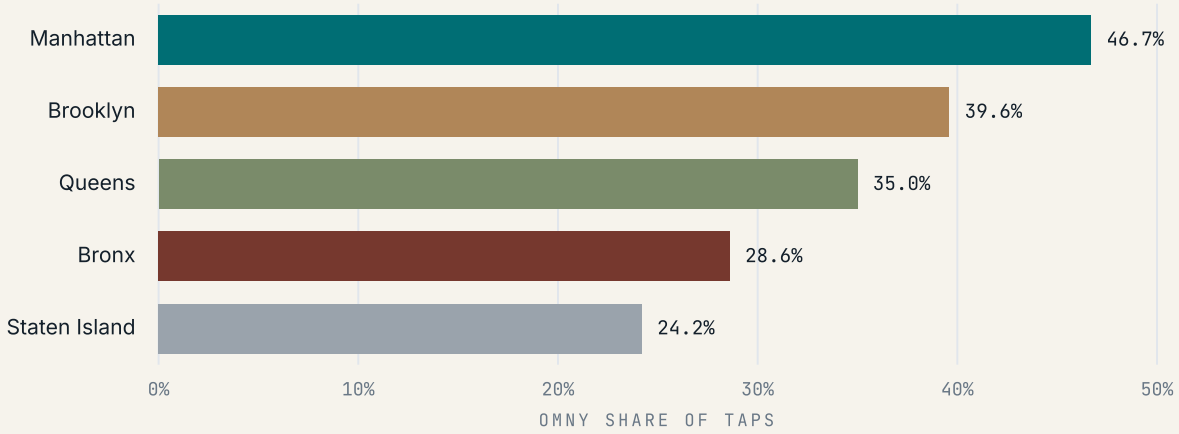

FIG 03 *OMNY share of taps by borough, from the EDA notebook's borough-level aggregation.*

Turning rows into features.

Time features

Hour-of-day, day-of-week, day-of-month, month, year, is_weekend, and is_holiday from a US federal holiday calendar.

Lag features

Three lags carry most of the signal: same station one hour ago (lag_1h), twenty-four hours ago (lag_24h), and one week ago (lag_168h). The week-ago lag implicitly carries hour-of-day, day-of-week, and station identity in a single value.

Rolling statistics

A 24-hour rolling mean and standard deviation per station, both shifted by one hour so they never see the future.

Categorical handling

All categoricals (`station_id`, `borough`, `hour`, `day_of_week`, `month`) go through scikit-learn's OneHotEncoder with `handle_unknown="ignore"`. The encoded matrix stays sparse to avoid memory blowup.

Magnitude standardization

Numeric features pass through StandardScaler. Tree models do not need this, but the pipeline applies it uniformly so the comparison is fair.

FINAL FEATURE SET

FEATURE	TYPE
station_id	one-hot
borough	one-hot
hour	one-hot
day_of_week	one-hot
month	one-hot
day	numeric
year	numeric
is_weekend	bool
is_holiday	bool
lag_1h	numeric
lag_24h	numeric
lag_168h	numeric
rolling_mean_24h	numeric
rolling_std_24h	numeric

Time-aware splitting to avoid leakage.

This is a time series. A random shuffle would let the model see training rows from after the test rows it was trying to predict, which inflates every metric. I used a chronological 80/20 train/test split. The last 20% of the timeline was held out as the final test set and was not used for model selection. Within the training portion, I used three expanding-window validation folds for model selection and backtesting.

Backtest folds

For backtesting I used a forward-chaining time-series cross-validation with three folds. Fold 1 trains on the earliest chunk and validates on the next chunk. Fold 2 extends training and slides the validation window forward. Fold 3 does the same again. The fold boundaries land at the row indices below, all on the modeling table after aggregation and lag features.

FOLD	TRAIN END	VAL START	VAL END	CALENDAR WINDOW (VAL)
1	3,017,153	3,017,153	3,620,583	≈ Q1–Q2 2024
2	3,620,583	3,620,583	4,224,013	≈ Q2–Q3 2024
3	4,224,013	4,224,013	4,827,443	≈ Q3–Q4 2024

Holdout test set

The final held-out test set is the last block of the data — 1,508,577 station-hour rows, the latest 20% of the timeline — and was never touched during model selection or hyperparameter search. I only ran the chosen final model against it once, at the very end, after backtesting was done.

The training data after fold 3's validation window (rows 4,827,443 to 6,034,307) was not used in any backtest fold but was included when fitting the final model on the full training set.

Six families of model, one clear winner.

I tested six model families plus a baseline to map the boundary between models that can and cannot capture nonlinear time effects.

The candidates

- **Dummy mean.** Predict the global mean. The performance floor.
- **Ridge regression.** Regularized linear model. Cheap baseline for what time features alone can do.
- **SGD regressor.** Same idea as Ridge, fit by stochastic gradient descent with L2 and elasticnet penalties.
- **Poisson regressor.** GLM with a log link, since ridership is a count. Underperformed due to overdispersion.
- **XGBoost.** Gradient-boosted trees, the workhorse of tabular regression.
- **LightGBM.** Leaf-wise gradient boosting, faster on data this large.

How I scored them

Four metrics together. **MAE** is in the same units as ridership. **RMSE** punishes big misses, which matters at peaks. **R^2** reports explained variance. **WMAPE** is a volume-weighted percentage error, the right thing to watch when station traffic ranges from a handful to several thousand riders per hour.

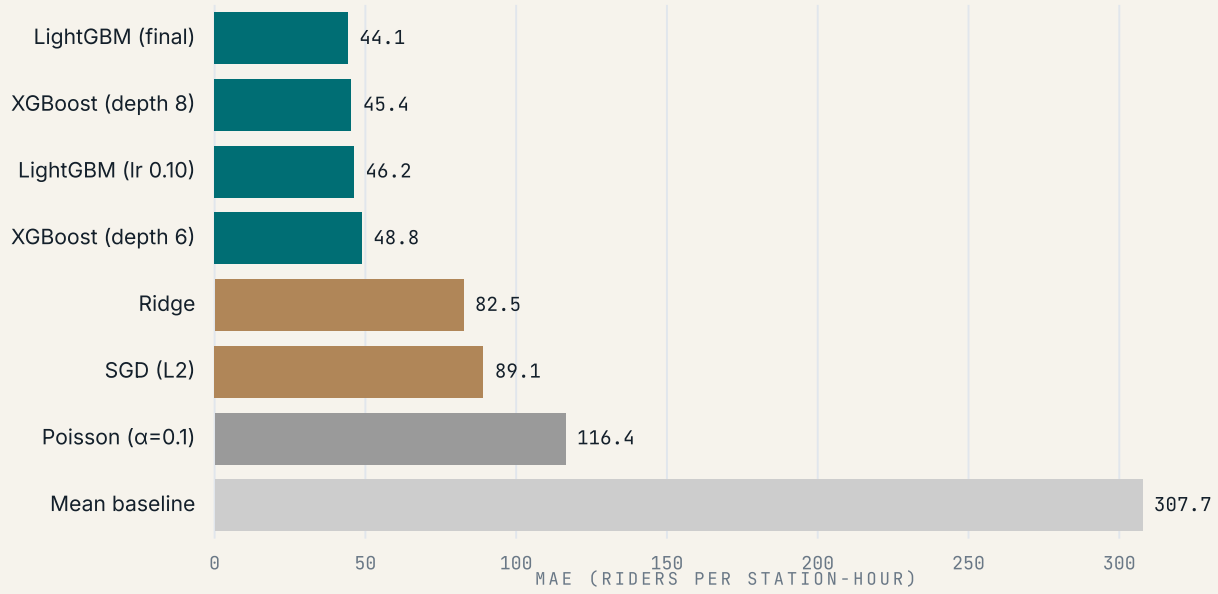

FIG 04 *Mean absolute error for the eight model configurations, averaged across the three backtest folds. Tree-based models cluster near 45 riders per station-hour; linear baselines sit closer to 85.*

The comparison table.

MODEL	MAE	RMSE	R^2	WMAPE
LightGBM · 127 leaves, lr 0.05, 300 trees	**44.14**	**125.64**	**0.9615**	**0.1441**
XGBoost · depth 8, lr 0.05, 300 trees	45.35	127.68	0.9602	0.1480
LightGBM · 63 leaves, lr 0.10, 200 trees	46.20	129.24	0.9592	0.1508
XGBoost · depth 6, lr 0.10, 200 trees	48.77	136.22	0.9547	0.1592
Ridge · α = 100	82.52	198.86	0.9037	0.2694
SGD · α = 1e-3, L2	89.12	202.40	0.9004	0.2900
Poisson · α = 0.1	116.36	506.93	0.3723	0.3795
Dummy (mean)	307.67	642.11	-0.0013	1.0043

Reading the table

Two findings stand out. First, every tree-based configuration sits at or above R^2 0.95, while linear configurations land closer to R^2 0.90. That gap is the cost of forcing a straight-line relationship onto a system that is fundamentally nonlinear. Second, Poisson regression, which I expected to do well because the target is a count, underperforms here. The poor Poisson results are consistent with overdispersion: ridership varies more than a simple count model can comfortably handle. A negative binomial model could be a useful follow-up comparison, but the tree models had already established a clear lead.

Holding up across time.

Validation results are encouraging, but they still represent one slice of time. The real question is whether the same model holds up when retrained with the window slid forward. So I ran three sequential backtest folds for the top configurations and tracked R^2 and MAE on each one.

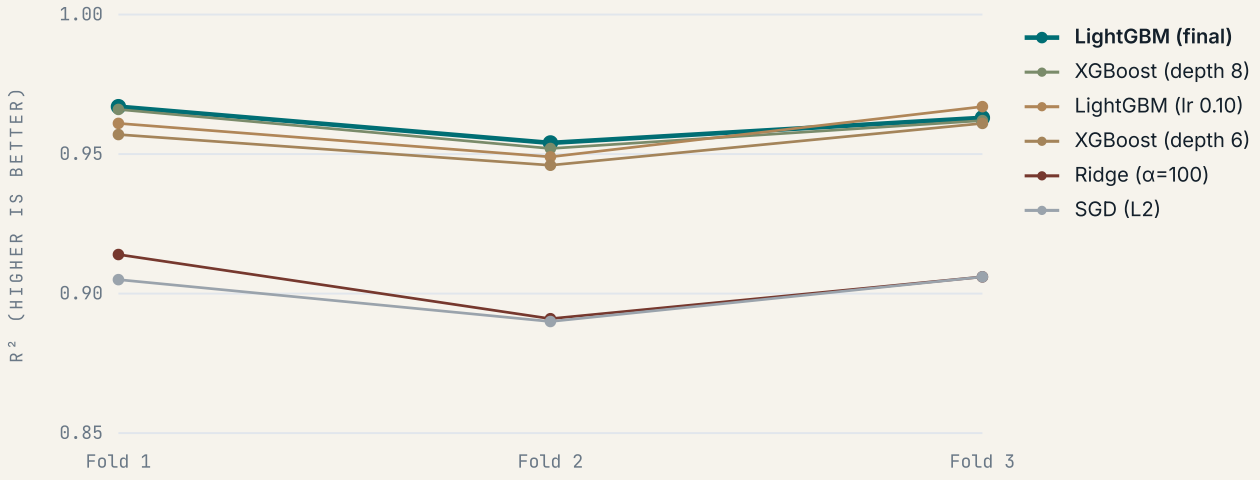

FIG 05 *R^2 across the three time-ordered backtest folds for the four tree configurations and the two linear baselines. LightGBM holds near 0.96; linear baselines sit near 0.90 with more variation across folds.*

What this tells me

The top LightGBM configuration averaged R^2 0.961 across the three folds with a standard deviation of 0.007, which is very small. Mean MAE was 44.1 riders per station-hour, with a standard deviation of about 4.7. The XGBoost runner-up tracks it within a percent or two on every fold. The linear baselines stay in the 0.90 region but show more variation across folds.

Sanity check on the year feature

I was concerned about including `year` as a feature, since the test set sits in a year that the training set may not have seen much of. So I retrained the top LightGBM and the top Ridge with and without the year feature. The deltas were small: LightGBM went from R^2 0.9615 to 0.9589 without year, and Ridge moved by less than 0.0001. Year stays in the final model because it does not hurt and it lets the model lean on the slow ridership recovery trend from 2022 to 2024.

The final scoreline.

36.4	94.5	0.981	11.5%
MAE · RIDERS / STATION-HOUR	RMSE	R^2 ON TEST	WMAPE

The final LightGBM (127 leaves, learning rate 0.05, 300 trees, min child samples 50) achieves an R^2 of 0.981 on the held-out test set with an MAE of 36.4 riders per station-hour. WMAPE is 11.5%, which means on average the model is off by about a tenth of the actual volume.

Predictions versus reality

The scatter sits tight against the diagonal across most of the volume range. The largest errors come from rare high-volume station-hours, where ridership can spike sharply above normal patterns. The model is conservative on some of those spikes, undershooting by a few hundred riders. That is also where RMSE picks up most of its weight.

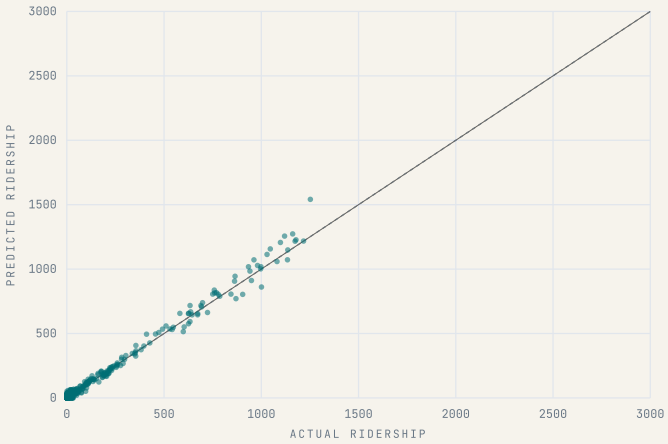

FIG 06 *Predicted versus actual ridership on the held-out test set, generated from the final model's notebook predictions. Each dot is one station-hour; the dashed line is perfect prediction.*

Where the errors live

The residuals are centered near zero, with larger errors concentrated in rare high-volume station-hours. The right tail is heavier than the left, consistent with the model undershooting some peak-hour cases rather than systematically overpredicting quieter periods.

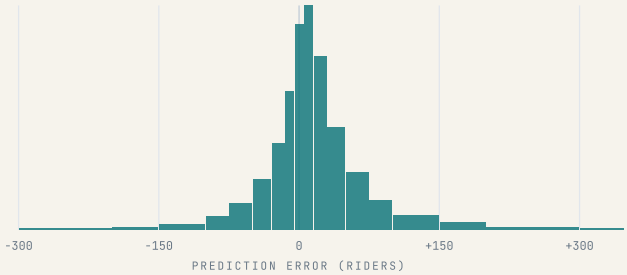

FIG 07 *Distribution of prediction errors on the test set, generated from the final model's notebook predictions.*